ST2195 COURSEWORK REPORT

PROGRAMMING FOR DATA SCIENCE

MARGARET JANELLE KAMTO
220657356

Table of Contents

Introduction 3
2.a Day and Time Delay Analysis 4
2.b Plane Age and Delay Correlation 6
2.c Logistic Regression Model 8

Introduction

Ten consecutive flight records from 1997 to 2006 were used and analyzed for the second part. This data was taken from Data Expo 2009: Airline on time data, Harvard Data verse. This dataset contains 29 features from the yearly flight data, which are year, month, day of month, day of week, departure time, scheduled departure time, arrival time, scheduled arrival time, unique carrier code, flight number, plane tail number, actual elapsed time, CRS Elapsed time, airtime, arrival delay, departure delay, origin code, destination code, distance, taxi in time, taxi out time, cancellation status, cancellation code, diverted status, carrier delay, weather delay, NAS delay, security delay, and late aircraft delay. Other than that, several supporting datasets such as plane data and airport data contains the details for each aircraft, manufacturing date, engine type; and the exact latitude and longitude for each airports. The analysis is centered on three primary issues which revolves around flight delays: The optimal the time and day to travel each year, the relationship between plane age and delays, and identifying the features that are the top contributing factors of diverted flights.

2.a Day and Time Delay Analysis

Ten years of raw data (1997-2006) in .csv was downloaded from https://doi.org/10.7910/DVN/HG7NV7 containing airline datas. This was then loaded and cleaned before any analysis was done.

Firstly, all yearly data was filtered to only show data which contains positive departure and arrival delays since negative delays do not count as "delays". Then, a new field is constructed called "TotalDelay" which contains the sum of departure and arrival delays for each flight. The data was loaded using the following code:

Py:

```
yearly_data = yearly_data[yearly_data['DepDelay'] > 0]
yearly_data = yearly_data[yearly_data['ArrDelay'] > 0]
yearly_data["TotalDelay"] = yearly_data['DepDelay'] + yearly_data['ArrDelay']
```

R:

```
filter(DepDelay > 0 & ArrDelay > 0) %>%
mutate(TotalDelay = DepDelay + ArrDelay) %>% ...
```

Both snippets of the code do the exact same thing, which is to filter out negative delays and create a new "TotalDelay" field. This was done with several justifications:

1. By not allowing negative delay to exist means the data is more representative for delays for each flight. This is because the positive delays does not cancel out the negative delays.
   Consider this example:
   - There exist 4 flights on Tuesday with these delays (in minutes) -> [10, -10, 5, -5]. By taking both the negative and positive datas, summing all the delays for Tuesday would result in a 0 minute delays, making it seem that the plane always arrive and depart on time, when it is extremely fluctuative in reality.

2. A "TotalDelay" field was constructed instead of using the delay's averages just for convenience purposes. Although it produces a number twice the average delay, this does not change the relative delay difference between each days, meaning the absolute order of each day group remains the same, and the same insight would be generated, regardless of which field is used.

By doing these processes. The data would look something like this:

	Year	DayOfWeek	ArrDelay	DepDelay	TotalDelay
1	1997	3	10.0	9.0	19.0
2	1997	4	5.0	11.0	16.0
3	1997	5	4.0	13.0	17.0
4	1997	2	44.0	38.0	82.0
7	1997	5	15.0	9.0	24.0

Figure 5 _Example data after preprocessing

Then, data from each year were grouped based on their departure day of week (1 = Monday, 7 = Sunday) and the mean total delay were calculated for each DayOfWeek for each year.

Results:

YEAR: 1997; Best day of the week to travel: 1
YEAR: 1998; Best day of the week to travel: 6
YEAR: 1999; Best day of the week to travel: 2
YEAR: 2000; Best day of the week to travel: 6
YEAR: 2001; Best day of the week to travel: 2
YEAR: 2002; Best day of the week to travel: 6
YEAR: 2003; Best day of the week to travel: 6
YEAR: 2004; Best day of the week to travel: 6
YEAR: 2005; Best day of the week to travel: 6
YEAR: 2006; Best day of the week to travel: 6

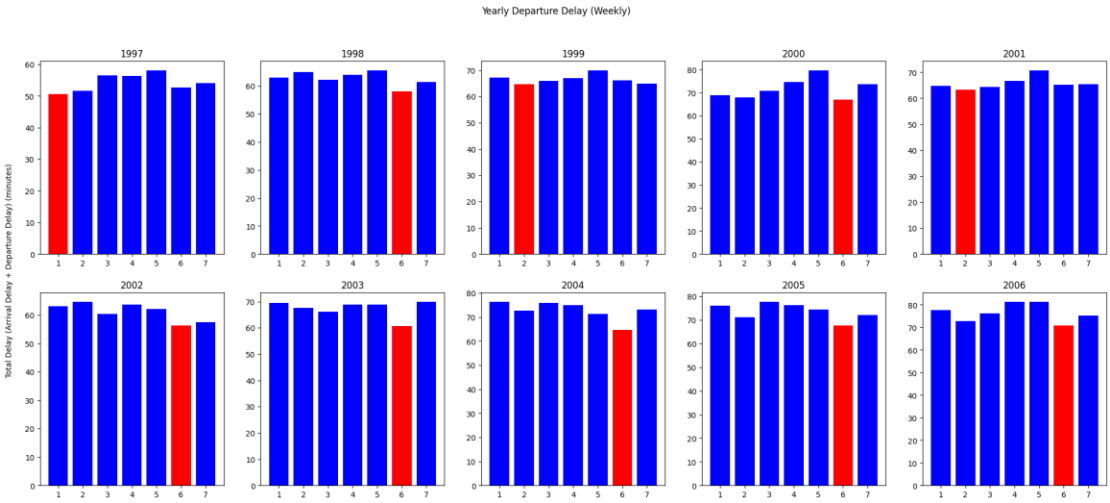

Figure 6 A bar plot showing the best day of the week to travel (based on total delay) for each year from 1997 to 2006

Based on the data, it can be concluded that **Saturday** consistently ranked as the best day of the week to travel with the lowest average delay in the past 5 years (2002-2006) and also ranked the lowest both in the year 2000 and 1998.
An analysis for the best time to travel is also done for each year, first by grouping the data into chunks of 1 hour. This is done by dividing the CSRDepTime by 100 to get the first 2 digits of the Departure Time. This was then added to a new field called "depTimeHour" using this code:

Py:

```
    current_year_data["depTimeHour"] = current_year_data["CRSDepTime"] // 100
```

R:

```
  df$depTimeHour = floor(df$CRSDepTime / 100)
```

Then, the same processes as the weekly analysis was conducted on the hour data.
Results:
YEAR: 1997; Best hour to travel: 5
YEAR: 1998; Best hour to travel: 5
YEAR: 1999; Best hour to travel: 3
YEAR: 2000; Best hour to travel: 24
YEAR: 2001; Best hour to travel: 5
YEAR: 2002; Best hour to travel: 0
YEAR: 2003; Best hour to travel: 1
YEAR: 2004; Best hour to travel: 4
YEAR: 2005; Best hour to travel: 9
YEAR: 2006; Best hour to travel: 1

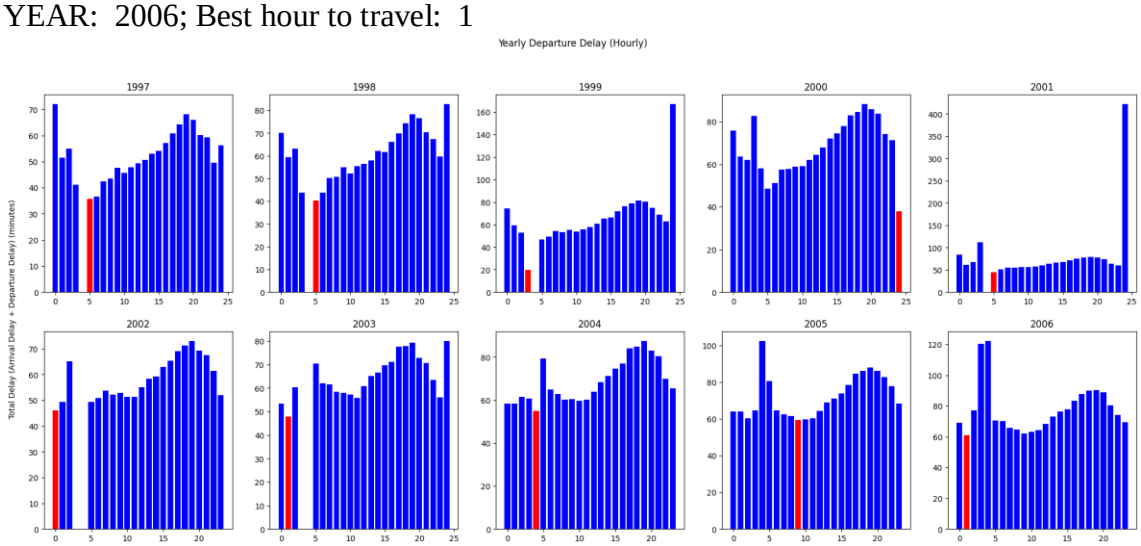

Figure 7 A bar plot showing the best hour of the week to travel (based on total delay) for each year from 1997 to 2006

The data shows that in three out of ten years (1997, 1998 and 2001) a departure time of 5 o'clock can give minimal delays. However, the trend doesn't hold up in recent years.

2.b Plane Age and Delay Correlation

To evaluate whether older planes suffer more delays on a year-to-year basis. **plane-data.csv** can be used in addition to the yearly flight data. This dataset provides us with the tail number, type, manufacturer, status, aircraft type, engine type, and most importantly the year.

The data is first cleaned using similar filter to 2(a) where only positive delays were considered. Then, using the combination of field "year" and "issue_date" (using the "year" field first, then filling NaNs and 0 with "issue_date".year), we can construct a new field called "plane_year" using the following code:

Py:

```
plane_data['issue_date'] = pd.to_datetime(plane_data['issue_date'])
plane_data['issue_year'] = plane_data['issue_date'].dt.year
plane_data['plane_year'] = plane_data['year'].fillna(plane_data['issue_year'])
plane_data['plane_year'] = np.where(plane_data['plane_year'] == 0, plane_data['issue_year'],
plane_data['plane_year'])
```

R:

```
plane_data <- plane_data %>%
  mutate(issue_date = as.Date(issue_date, format = "%m/%d/%Y")) %>%
  mutate(issue_year = year(issue_date))

plane_data <- plane_data %>%
  mutate(plane_year = ifelse(!is.na(year), as.numeric(year), issue_year))

plane_data <- plane_data %>%
  mutate(plane_year = ifelse(plane_year == 0, issue_year, plane_year))
```

Essentialy, "plane_year" is obtained from either the "year" field from plane-data.csv or the year value from the "issue_date" field in that order of priority in case null values exist in the former field.

Next, the yearly data and plane data were merged into a single dataset using the "TailNum" field as the merging key. After merging, all the year data was converted into ints to make age calculation easier. A new field called "Plane_age" is also constructed by subtracting the plane year from the departure year.

Py:

```
    year_data = year_data.merge(plane_data, on='TailNum', how='left')
    year_data['plane_age'] = year_data['Year'] - year_data['plane_year']
    year_data = year_data[year_data['plane_age'] >= 0]
```

R:

```
  year_data <- left_join(year_data, plane_data, by = "TailNum")
  year_data <- mutate(year_data, plane_age = Year - plane_year)
  year_data <- filter(year_data, plane_age >= 0)
```

The code also filters out wrong data by only taking the rows with positive plane age since several dirty and misinputed data exist which causes negative plane age to exist. Resulting in this clean data can be used for graphing and analysis.

	Year	TailNum	TotalDelay	issue_date	year	issue_year	plane_year	plane_age
53	1997	N128DL	280.0	1988-07-13	1988.0	1988.0	1988	9
54	1997	N125DL	142.0	2008-02-06	1988.0	2008.0	1988	9
55	1997	N656DL	77.0	1990-03-09	1990.0	1990.0	1990	7
56	1997	N662DN	71.0	1991-02-15	1991.0	1991.0	1991	6
57	1997	N614DL	45.0	1986-02-25	1985.0	1986.0	1985	12
...	...	...	...	...	...	...	...	...
1756666	1997	N278AA	44.0	1985-08-08	1985.0	1985.0	1985	12
1756667	1997	N448AA	19.0	2004-03-12	1987.0	2004.0	1987	10
1756668	1997	N410AA	103.0	1986-05-27	1986.0	1986.0	1986	11
1756669	1997	N431AA	204.0	1987-09-02	1986.0	1987.0	1986	11
1756671	1997	N545AA	19.0	1995-06-12	1976.0	1995.0	1976	21

Figure 8 Preprocessed data after cleaning, merging, and filtering processes

Then, by iterating through the years and grouping the data based on the plane age, we can calculate the mean TotalDelay from each year for each plane age. The result is as follows:

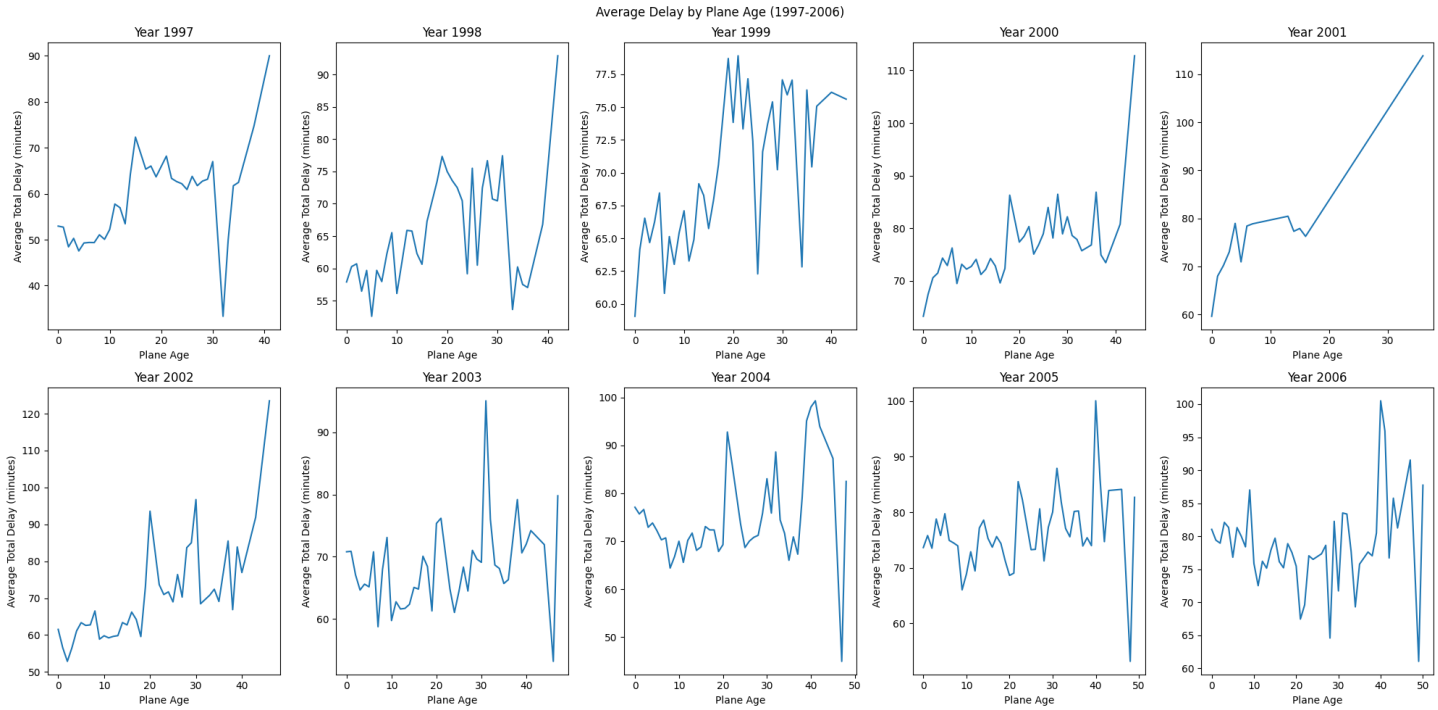

Figure 9 A line graph showing average total delay vs plane age for each year from 1997 to 2006

From the data above. A positive trend in delays emerges as plane age increases with lower delay for plane age closer to 0, and higher the older the plane gets. This was more apparent in some years than others. Even though delays fluctuate across plane ages, it can clearly be said that there is a positive correlation between plane age and total delays.
\

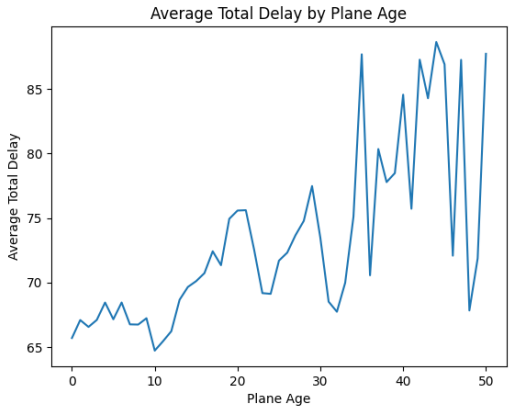

Figure 10 A line graph showing the aggregated average total delay vs plane age

Aggregating all the years also shown some positive correlation in terms of delays vs plane age with lower delays on a newer plane (plane age approaching 0) and more extreme and more fluctuative delays on older planes. A balanced dataset is key to determining whether certain factors correlate highly to the number of diverted flights. Following this mindset, the first step of this problem requires us to balance the dataset by counting the number of diverted flights each year, and randomly sampling the same amount of non-diverted flights in the same year. This was done for each of the year using the following code:

Py:

```
diverted_year_data = yearly_data[yearly_data['Diverted'] == 1]
non_diverted_year_data = yearly_data[yearly_data['Diverted'] == 0]
.sample(n=diverted_year_data.shape[0])
```

R:

```
diverted_year_data <- filter(yearly_data, Diverted == 1)
non_diverted_year_data <- sample_n(filter(yearly_data, Diverted == 0), nrow(diverted_year_data))
```

2.c Logistic Regression Model

A balanced dataset between positive and negative classes allows us to remove some major biases in the training process. This bias usually occur when certain class occurs a lot more than the other, causing the training model to lean into a certain answer more frequently due to high evaluation performance score.

For more accurate model, **airport-data.csv** was also loaded into the program to provide new fields such as airport names and the latitude and longitude of each airports. These fields could help the model to converge and predict better.
After loading the airport data, a merging step was done to join yearly data and the airport data. This was done to get both the latitude and longitude for both origin and destination airport.

Py:

```
    complete_data = data.merge(airport_data, left_on='Origin', right_on='iata', how='left')
    complete_data = complete_data.merge(airport_data, left_on='Dest', right_on='iata', how='left',
suffixes=('_origin', '_dest'))
```

R:

```
complete_data <- balanced_year_datas[[i]] %>%
  left_join(airport_data, by = c('Origin' = 'iata'), suffix = c("", "_origin")) %>%
  left_join(airport_data, by = c('Dest' = 'iata'), suffix = c("_origin", "_dest"))
```

	Year	Month	DayofMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	...	country_origin	lat_origin	long_origin	iata_dest	airport_dest	city_dest	state_dest	country_dest	lat_dest	long_dest
0	1997	1	13	1	1142.0	1020	NaN	1130	WN	1293	...	USA	35.040222	-106.609194	PHX	Phoenix Sky Harbor International	Phoenix	AZ	USA	33.434167	-112.008056
1	1997	1	4	6	1505.0	1415	NaN	1535	WN	743	...	USA	30.194533	-97.669872	PHX	Phoenix Sky Harbor International	Phoenix	AZ	USA	33.434167	-112.008056
2	1997	1	6	1	1256.0	1300	NaN	1505	DL	159	...	USA	39.046143	-84.662172	PHX	Phoenix Sky Harbor International	Phoenix	AZ	USA	33.434167	-112.008056
3	1997	1	7	2	805.0	630	NaN	740	WN	1710	...	USA	31.806667	-106.377806	PHX	Phoenix Sky Harbor International	Phoenix	AZ	USA	33.434167	-112.008056
4	1997	1	26	7	942.0	935	NaN	1311	CO	1915	...	USA	40.692497	-74.168661	PHX	Phoenix Sky Harbor International	Phoenix	AZ	USA	33.434167	-112.008056
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
24157	1997	8	11	1	2201.0	2205	2310.0	2315	UA	2080	...	USA	33.942536	-118.408074	SFO	San Francisco International	San Francisco	CA	USA	37.619002	-122.374843
24158	1997	5	28	3	836.0	840	1113.0	1121	US	2200	...	USA	39.871953	-75.241141	PBI	Palm Beach International	West Palm Beach	FL	USA	26.683162	-80.095594
24159	1997	12	3	3	659.0	700	1238.0	1231	DL	1226	...	USA	40.788388	-111.977773	ATL	William B Hartsfield-Atlanta Intl	Atlanta	GA	USA	33.640444	-84.426944
24160	1997	4	12	6	1221.0	1200	1817.0	1806	AA	1870	...	USA	33.942536	-118.408074	BNA	Nashville International	Nashville	TN	USA	36.124477	-86.678182
24161	1997	3	2	7	1458.0	1453	1630.0	1630	AS	452	...	USA	45.588722	-122.597500	SFO	San Francisco International	San Francisco	CA	USA	37.619002	-122.374843

Figure 11 Preprocessed data after merging

For easier and faster training process to converge, most fields were omitted, leaving only 10 crucial fields which are:

1. Month – The flight departure month
2. DayofMonth – The flight departure day of the month
3. CRSDepTime – The scheduled departure time
4. CRSArrTime – The scheduled arrival time
5. Distance – distance between origin airport and destination airport in miles
6. Diverted – whether the flight was diverted or not (1=yes, 0=no) (This will be the label)
7. lat_origin – the origin airport's latitude
8. long_origin – the origin airport's longitude
9. lat_dest – the destination airport's latitude
10. long_dest – the destination airport's longitude.

Since the model would be trained to determine if a flight is diverted or not, the "Diverted" field was considered as a label (y) and the rest were features (X).

Py:

```
for i, data in enumerate(complete_datas):
    X, y = data.drop('Diverted', axis=1), data['Diverted']
```

R:

```
X = select(data, -Diverted)
y = data$Diverted
```

After separating the features from the label, a normalization step is done to the features to minimize bias as much as possible. This was done using Z-Score Normalization, which makes each column have a mean of 0 and standard deviation of 1.

$$X' = \frac{X - \mu}{\sigma}$$

Where:
X' = The feature new value
X = The feature old value
μ = Mean of the feature
σ = Standard deviation of the feature

Then, the logistic regression model is trained by using the SKlearn library in python, and tidyverse in R.
Essentially, logistic regression aims to find several coefficients associated with the variables using odds ratio S where

$$S = \frac{p}{1-p}$$

Where p is the probability of success and 1-p is the probability of failure. P itself is a squished linear regression equation using the sigmoid function

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots \beta_n x_n)}}$$

After plugging the value of p and simplifying, we find that

$$\ln(S) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Where x is the features and β is the coefficient for each feature.

The coefficients of each feature is extracted after the model has finished training, This was then plotted for each year (1997-2006) with the following as the result.

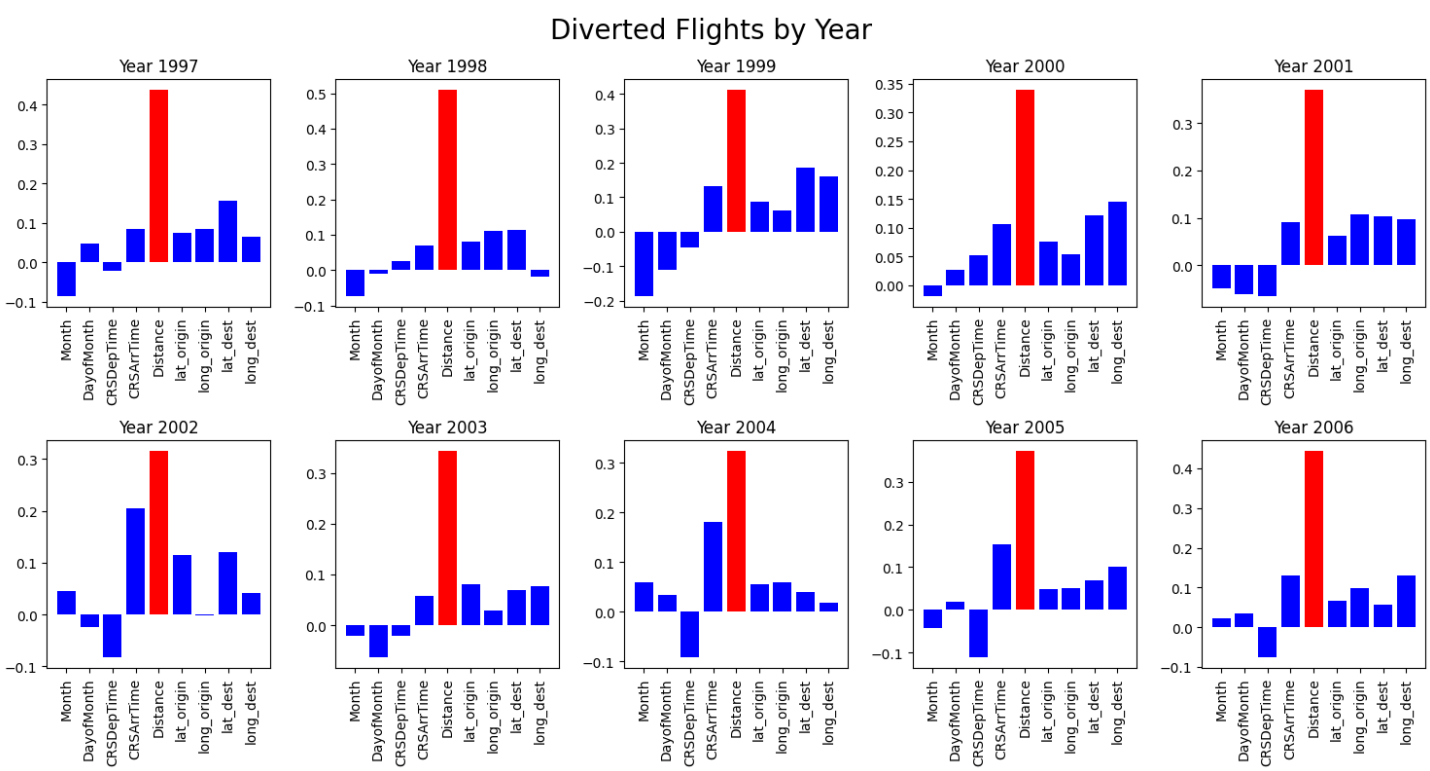

Figure 12 A bar plot showing each feature's coefficients

From the above data, it can be concluded that the "Distance" field is the most highly correlated with the diverted flights, getting the highest coefficients across all years from 1997 to 2006, making it the single biggest factor in determining whether a flight would be diverted or not.